

12306-mcp 小白指南

一个 2200+ Star 的开源 MCP 服务器：让 Claude 等大模型直接用自然语言查 12306 车票、中转和经停站。

学习来源：github.com/Jooooook/12306-mcp (MIT, 快照 commit ff6439d, 2026-10-07 克隆实测)

开源项目类

MCP 工具链

疑问式目录

真实查询实验



疑问目录

Q1 这是什么

Q2 什么是 MCP

Q3 整体架构

Q4 启动时做什么

Q5 八个工具

Q6 怎么调 12306

Q7 中转与经停

Q8 怎么装

Q9 给我什么能力

Q10 动手实验

Q1

这个项目是什么？解决什么问题？

12306-mcp 是作者 Joook 用 TypeScript 写的一个**车票查询服务器**。矛盾在于：12306 官方只给人看的网页/App，**没有给大模型用的 API**；而大模型想帮你“查后天去北京的高铁”，必须有人替它去网页背后抓数据。这个项目就是那个“替身”——它把 12306 的网页接口包装成 8 个标准工具，任何支持 MCP 的 AI 客户端（Claude Desktop、Claude Code、Codex 等）配一行就能用。

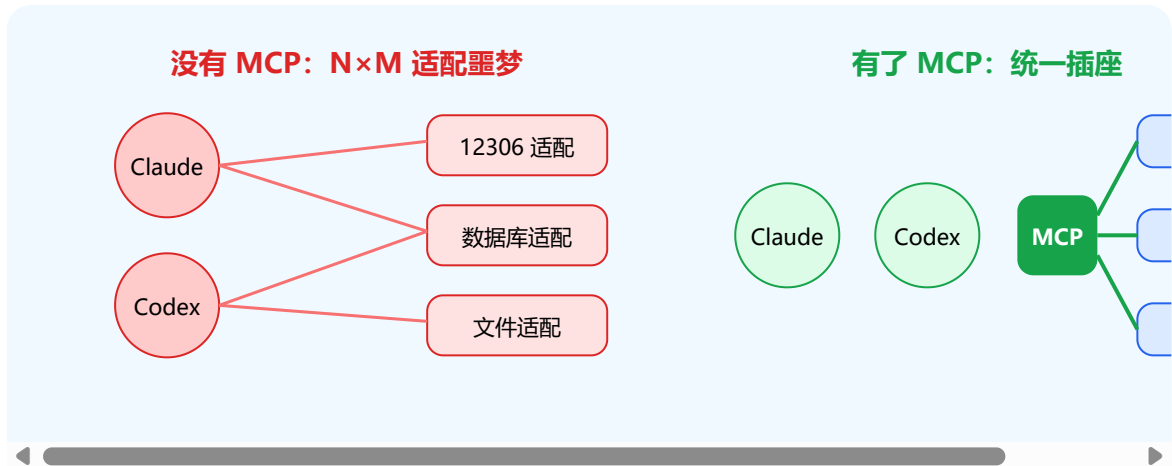
一句话：12306-mcp = 12306 网页接口 → MCP 标准工具的**翻译器**。它只查票不买票（订票要登录、验证码、支付，属于另一个世界）。

功能	状态	对应工具
查余票（车次/时间/价格/余票）	✓	get-tickets
中转查询（指定中转站）	✓	get-interline-tickets
经停站查询	✓	get-train-route-stations
城市↔车站代码换算	✓	4 个基础工具

Q2

什么是 MCP？为什么需要它？

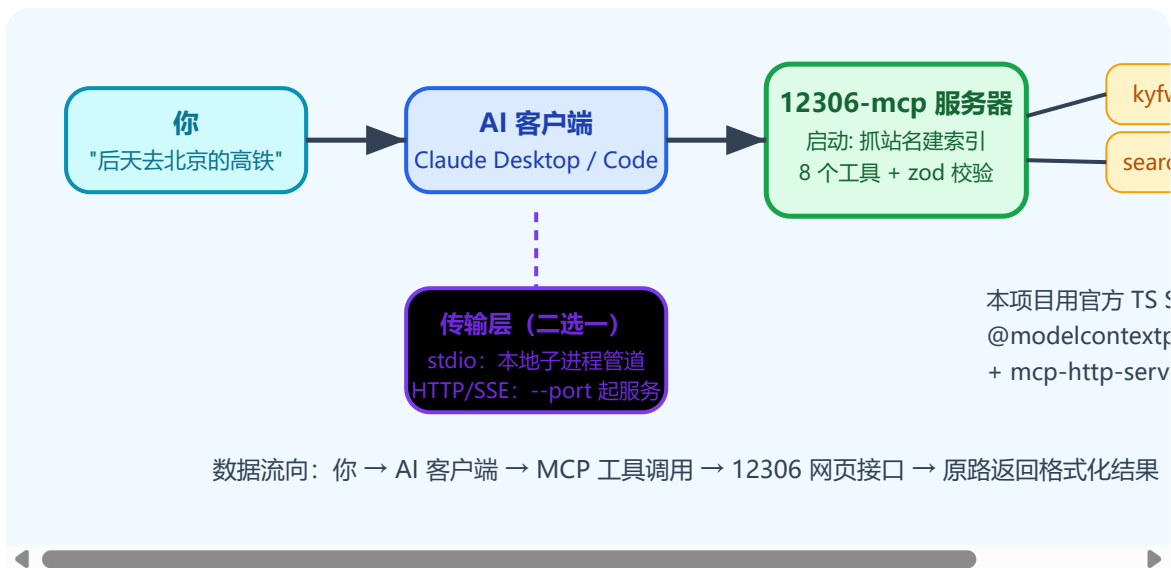
MCP (Model Context Protocol, 模型上下文协议) 是 Anthropic 2024 年提出的开放协议：规定"AI 应用如何发现和调用外部工具/数据源"。可以类比 **USB-C 接口**：以前每个外设一种插头（每个插件写一套自定义接入），现在统一一个口，键盘、显示器、硬盘插上就能用。



一次 MCP 会话分三步：① **握手**（客户端问"你是谁、有哪些工具"）→② **看菜单**（服务器返回工具清单，每个工具附名字、说明、参数 schema）→③ **点菜**（客户端以 JSON 调用工具，拿回文本/数据结构）。12306-mcp 是 **工具提供方 (server)**，Claude Desktop 这类是 **调用方 (client)**。

Q3

整体架构长什么样？



源码极精简：核心就 `src/index.ts` (1633 行) + `src/types.ts` (315 行)。两种运行形态：`stdio` (客户端把服务器当本地子进程拉起，用标准输入输出传 JSON，适合桌面客户端) 和 `HTTP` (`--port` 起 SSE/Streamable HTTP 服务，适合远程或多客户端共享)。

Q4

服务器启动时偷偷做了什么？

启动不是"监听端口就完事"，它先做三件准备工作（实测启动到就绪约几秒，取决于网络）：



含义：车站代码（如北京=BJP）是 12306 接口的"护照"，LLM 只会说"北京"，索引表负责翻译

- **站名数据**：从 12306 首页 HTML 里找到站名 JS 文件路径，下载解析出全国几千个车站，再手工补充缺失站（如"成都东"）。
- **四张索引表**：**STATIONS**（代码→车站全信息）、**CITY_STATIONS**（城市→市内所有站）、**CITY_CODES**（城市→同名代表站）、**NAME_STATIONS**（站名→代码）。
- **路径探测**：12306 的余票查询路径会不定期变化（如 **leftTicket/queryA** ↔ **queryG**），启动时先请求 init 页探测当前有效路径，**这是项目能"活得久"的关键设计**。

Q5

八个工具怎么分工？为什么要分层？



分层是刻意的：**大模型不会背车站代码**。查票前它必须先调基础工具把"北京"翻译成 BJP、"上海"翻译成 SHH，再拿代码去调核心工具。这种"先查字典再办事"的模式，是所有数据查询类 MCP 服务器的通用套路（你的 agent 知识库多处可见同类设计）。

Q6

它具体怎么把 12306 网页接口"骗"成 API 的?

12306 网页自己也要调后端 JSON 接口 (如 `/otn/leftTicket/query`) , 只是这些接口**不公开、要带 Cookie、路径还会变**。12306-mcp 的做法:

1. **参数校验**: 每个工具用 `zod` 声明 schema (如日期必须 10 位 `yyyy-MM-dd`、车站代码必须存在、筛选标志只能是 GDZTKOFS 中的字母) , 非法输入在进 12306 之前就被打回——**这是 LLM 工具的守门员**。
2. **Cookie 预处理**: 先访问 init 页拿到会话 Cookie, 再带 Cookie 请求查询接口 (12306 对裸请求会拒绝) 。
3. **中文名自动翻译**: `get-tickets` 的出发/到达参数直接收"北京"这种中文, 内部查索引表转代码; 实验实测传中文名成功。
4. **结果格式化**: 把 12306 返回的密麻字段 (`secretStr`、`yz_num` 等) 翻译成"G1 北京南→上海虹桥 06:30→11:24 二等座 剩余1张票 661元"这种人话, 再喂回大模型。

值得偷的设计: 查询路径启动时动态探测 (Q4 ③) + 输入输出双层翻译 + zod 守门。写任何"包装网页服务"的工具都适用。

Q7

中转查询和经停查询有什么特别的?

中转 (get-interline-tickets) : 走 12306 的 `/lcquery/queryU` 接口 (网页版"中转换乘"页背后的接口) , 可指定中转站, 结果只取前 10 条——作者在文档里明确写了这个限制, **诚实的边界标注是开源项目的好品味**。

经停 (get-train-route-stations) : 先从 `search.12306.cn` 按车次号+日期查到列车完整编码, 再格式化全程停靠表。实验实测 G531 次返回 13 个经停站的到发时刻 (见 Q10) 。

Q8

怎么把它装进自己的环境?

方式一：一行 npx (stdio, 适合 Claude Desktop / Claude Code)

```
npx -y 12306-mcp
```

方式二：HTTP 服务模式（远程或多客户端）

```
npx -y 12306-mcp --port 8080
```

方式三：Docker

```
docker build . -t 12306-mcp && docker run -p 8080:8080 -d 12306-mcp npx 12306-mcp --port 8080
```

Claude Desktop 配置 (`claude_desktop_config.json` 里加) :

```
"mcpServers": {  
  "12306-mcp": { "command": "npx", "args": ["-y", "12306-mcp"] }  
}
```

环境要求：Node.js ≥ 18 ；开发调试用的 MCP Inspector 2.0 要 Node ≥ 22.19 。本机实测 Node v24.21.0 通过。

这个项目给我什么（能力清单）

① 一句话查票

在 Claude Desktop 里说“后天上午北京到上海的高铁，二等座有票的”，AI 自动完成日期计算、站名翻译、查票、汇总。

② 自己写 MCP 服务器的范本

1600 行 TypeScript 展示了 MCP 工具注册、zod 校验、双传输模式（stdio/HTTP）的完整写法，可复制为模板。

③ “包装网页接口”方法论

动态路径探测 + Cookie 预处理 + 输入

④ 与你的 agent 知识库互相印证

工具分层、依赖注入、边界标注等设计

输出翻译，适用于任何没有公开 API 但要被 AI 使用的网站。

在 agent-harness 等已学项目中同构，可互相类比巩固。

⑤ 出行规划自动化素材

中转查询 + 经停查询组合，可做成“每周五自动查回家车票”类的定时 agent 工具。

⑥ 边界清醒

只查不买；中转只回前 10 条；余票数据以 12306 实时为准但官方随时可能改接口——本项目可能哪天就失效，这是这类工具的宿命。

Q10

动手实验：五步真实查询（全部实测）

实验材料在 `12306_mcp/exercise/`：`mcp_client.mjs` 是一个最小 MCP 客户端，通过 stdio 启动 `repo/build/index.js` 并连续调用 5 个工具。以下输出为本机 2026-10-07 真实运行结果（完整记录见 `exercise/run_output.txt`）。

第 1 步 · 编译项目

```
cd 12306_mcp\repo
npm i --ignore-scripts      :: 跳过生命周期脚本，依赖装完
npx tsc                    :: 手动编译，生成 build\index.js
```

✅ 预期结果（实测）：`build\index.js` 生成，无编译错误。

第 2 步 · 启动客户端看工具菜单

```
cd 12306_mcp\exercise
npm i --ignore-scripts
node mcp_client.mjs
```

目的：验证 MCP 握手与工具发现。服务器启动时会实时抓取 12306 全国站名（几秒）。

✅ 预期结果（实测）：打印 8 个工具（`get-current-date / get-station-code-* / get-tickets / get-interline-tickets / get-train-route-stations`）。

第 3 步 · 城市名翻译 + 查明天高铁票

客户端依次调用 `get-current-date` → `get-station-code-of-citys("北京|上海")` → `get-tickets(明天, 北京, 上海, G)`。

实测输出（节选）：

```
{"北京":{"station_code":"BJP"},"上海":{"station_code":"SHH"}}
```

G1 北京南 -> 上海虹桥 06:30 -> 11:24 历时：04:54

- 商务座：剩余8张票 2315元

- 一等座：剩余2张票 1058元

- 二等座：剩余1张票 661元

（共 6521 字符的完整车次表）

✅ 预期结果：返回真实余票（G1 二等座仅剩 1 张）。传中文城市名直接被接受。

第 4 步 · 查经停站

实测输出（G531 次，节选）：

1|北京南|---|06:08

2|天津南|06:39|06:41

...

10|南京南|10:42|10:45

...

13|上海虹桥|12:04|12:04

✅ 预期结果：13 个经停站到发时刻完整返回。

第 5 步 · 改参数再玩（自主探索）

```
:: 把 mcp_client.mjs 里的 trainFilterFlags 改成 'D'（动车）  
:: 或把 fromStation/toStation 改成你自己的常用路线，重跑  
node mcp_client.mjs
```

✅ 预期结果：车型筛选生效，只返回 D 字头车次；换城市同样成立。

实验中踩到的两个真实坑（写进了代码注释）：① Windows 下

`import.meta.url.pathname` 会得到 `/F:/...` 造成路径重复，必须用 `fileURLToPath`；② 工具参数名以 zod schema 为准（`citys`、`trainCode`、`departDate`），文档与 schema 不一致时以 schema 为准。

来源与核实：触发记录为 @ai_fengshou 的 X 推荐帖（帖称 1.8k star，实测 2258 仍在涨）；上游 github.com/Joook/12306-mcp，MIT 许可，快照 commit ff6439d（2026-10-07 浅克隆）。全部实验命令在本机 Windows 11 + Node v24.21.0 真实执行，输出存于 exercise/run_output.txt。

与已有知识库的关联：MCP 协议与 agent-harness / agent-engineering 等项目的工具接入同构；zod 参数守门对应“证据优先质检”的输入侧思维；工具分层模式可与已学概念互相[[链接]]（详见知识提案）。

局限：只读查询、不含订票；依赖 12306 网页接口的现行结构，官方改版即可能失效；中转结果只回前 10 条；余票为查询时刻快照。